

Statiska metoder och medlemmar

Då vi hittills definierat klasser har vi för det mesta även haft en eller flera datamedlemmar i klasserna. Dessa datamedlemmar finns sedan instantierade i ett objekt. Varje objekt har en egen version av varje medlem. Inget data är delat mellan olika instanser av samma objekt. I vissa fall vore det praktiskt om alla instanser av samma klass kunde dela samma data så att man t.ex. inte behöver flera instanser av en minneskrävande klass. Ett bra exempel då man vill ha en delad variabel är när man vill ge en unik id till alla instanser av en viss klass. Vi kan t.ex. titta på klassen `Product`:

```
#include <iostream>
#include <string>

class Product {
public:
    // konstruktor
    Product (const string & Name, int Id) { m_Name = Name, m_Id = Id; }

    // skriv ut produkten på skärmen
    void print () const;

private:
    string m_Name;
    int    m_Id;
};

void Product::print () const {
    cout << "Produktnamn: " << m_Name << ", id: " << m_Id << endl;
}

int main () {

    Product * Lager [10];
    int Id = 0;
    string Names [10] = {"a", "b", "c", "d", "e", "f", "g", "h", "i", "j"};

    // skapa nya produkter
    for ( int Index = 0; Index < 10; Index++ ) {
        Lager[Index] = new Product ( Names[Index], Id++ );
    }

    // skriv ut vårt lager
    for ( int Index = 0; Index < 10; Index++ ) {
        Lager[Index]->print ();
    }
}
```

Vi använder en lokal variabel `Id` i `main()` för att numrera våra produkter. Egentligen är denna id produktspecifik data, och borde egentligen vara i klassen `Product`. Men hur vi än försöker kan inte klassen `Product` veta vilken som är den senaste använda id:n. Lösningen är att skapa en *statisk variabel* i klassen `Product` som innehåller den senaste id:n. På så vi kan varje instans som skapas läsa denna variabel, kopiera värdet till sin medlem `m_Id` och inkrementera den statiska variabeln. En statisk variabel deklareras med nyckelordet `static`. Vi modifierar `Product` en aning:

```
#include <iostream>
```

```

#include <string>

class Product {
public:
    // konstruktor
    Product (const string & Name);

    // skriv ut produkten på skärmen
    void print () const;

private:
    // en statisk variabel
    static int m_NextId;

    // normala medlemmar
    string m_Name;
    int    m_Id;
};

Product::Product (const string & Name) {
    m_Name = Name;
    m_Id = m_NextId++;
}

// initialisera vår statiska variabel
int Product::m_NextId = 0;

void Product::print () const {
    cout << "Produktnamn: " << m_Name << ", id: " << m_Id << endl;
}

```

Vi har nu en privat statisk variabel. den är privat så att inte externa entiteter skall kunna modifiera nästa id, och på så vis eventuellt skapa produkter med samma id. Man måste ge ett initialvärde åt statiska medlemmar, men det kan inte göras inom `class`-definitionen. Man måste då minnas att en `class` endast är en ritning för hur egentliga instanser skall se ut, och inte reservera minne åt medlemmar eller något sådant. Så minne för `m_NextId` kan inte reserveras inne i klassen, det görs en gång (och endast en gång) då den första instansen av klassen skapas. Initialiseringen sker alltså endast en gång! Vi kan sedan använda variabeln inne i klassens metoder som vilken medlem som helst, bara vi minns att om vi ändrar på dess värde syns samma ändring i alla andra instanser av klassen. Vi kan nu modifiera vår `main()` så den använder den nya klassen:

```

int main () {
    Product * Lager [10];
    string Names [10] = {"a", "b", "c", "d", "e", "f", "g", "h", "i", "j"};

    // skapa nya produkter
    for ( int Index = 0; Index < 10; Index++ ) {
        Lager[Index] = new Product ( Names[Index] );
    }

    // skriv ut vårt lager
    for ( int Index = 0; Index < 10; Index++ ) {
        Lager[Index]->print ();
    }
}

```

Nu har vi lyckats förflytta den för `main()` irrelevanta hanteringen av id:n till `Product`.

Statiska medlemmar förstörs då programmet terminerar. Trots att vi skulle radera varje instans av vår `Product`-klass skulle `m_NextId` ändå hålla samma värde.

Statiska metoder

Förutom endast statiska medlemmar i klasser och variabler i funktioner kan man även ha *statiska metoder* i klasser. Vad gör då en statisk metod? En statisk metod är en metod som tillhör klassen med som inte tillhör något instantierat objekt. Den kan inte accessera normala medlemmar som hör till instanser av klassen, utan endast statiska medlemmar. Som exempel kan vi göra en statisk metod till `Product` som skriver ut vad nästa id skall vara:

```
class Product {
public:
    ...

    // statisk metod
    static void printId () { cout << "Nästa id: " << m_NextId << endl; }

    ...
};
```

En statisk metod definieras genom att placera nyckelordet `static` framför metodens returvärdesdefinition. Metoden kan implementeras antingen som vi gjort ovan direkt inline eller så på normalt sätt:

```
void Product::printId () {
    cout << "Nästa id: " << m_NextId << endl;
}
```

Då man använder metoden kan man inte referera till den via ett objekt, utan direkt via klassen:

```
Product P ("Foo");
P.printId ();          // fungerar inte!
Product::printId ();  // korrekt anrop
```

Singleton

Vad har man då för nytta av statiska metoder? I fallet ovan är nyttan klar, men i övrigt finns det inte så väldigt många situationer där statiska metoder är nödvändiga. Ett lysande undantag är en *Singleton*-klass. En dylik klass är en klass som man vill att skall finnas i endast en instans i ett program. Det kan t.ex. vara frågan om en klass som abstraherar en resurs som det finns nedast en av. Istället för att skapa en klass och placera den någonstans så att alla andra klasser som behöver den kan komma åt den, kan man göra den till en Singleton. Det är ett väldigt enkelt och elegant sätt att uppnå en lösning på problemet. Läs mera om Singletons och andra praktiska metoder i *Design Patterns* (se [Referenser](#)).

[Förutgående](#)

Standardvärden för funktioner och metoder

[Hem](#)

[Upp](#)

[Nästa](#)

Pekaren `this`